1.INTRODUCTION

1.1 ORGANIZATION PROFILE

(*KICE Infosystems* -Software Development Division)

KICE Infosystems – Software Development Division is a professional IT solutions and software development company headquartered in Tirupur, Tamil Nadu, with operational presence in Coimbatore. The organization is ISO 9001:2015 certified and focuses on delivering high-quality web and software development services to businesses across multiple industries.

The development division specializes in customized software development, web application development, website design, and enterprise solutions. Our team of experienced developers and technology professionals works closely with clients to design scalable, secure, and efficient digital systems that improve operational efficiency and business productivity.

KICE Infosystems provides end-to-end software development services, including requirement analysis, system architecture design, UI/UX development, coding, testing, deployment, and long-term maintenance support. We also assist organizations in digital transformation initiatives, enabling them to modernize their operations through innovative technology solutions.

Our development expertise includes technologies such as Python, Flask, Django, PHP, MERN Stack, .NET Technologies, Java, MySQL, MongoDB, and modern web frameworks. We develop a wide range of applications including business management systems, e-commerce platforms, enterprise web applications, and customized digital solutions for startups, SMEs, and enterprises.

The Software Development Division follows industry-standard development practices and quality assurance procedures to ensure reliable and high-performance solutions. With a strong commitment to innovation, quality, and customer satisfaction, KICE Infosystems continues to support organizations in building powerful digital platforms that deliver measurable business value.

(KICE Infosystems – Software Training and Research Development Institute)

KICE Infosystems – Software Training and Research Development Division is a leading professional training institute established in Tirupur with branches in Coimbatore. The institute is **ISO 9001:2015 certified** and affiliated with **BSSVE (Bharat Sevak Samaj Vocational Education), New Delhi**, a national development agency established by the Planning Commission of India.

The training division focuses on providing industry-oriented technical education, research guidance, and skill development programs for students, graduates, and professionals. Our goal is to bridge the gap between academic learning and real-world industry requirements by offering practical and project-based training.

KICE Infosystems provides training in a wide range of modern technologies including Python Programming, Full Stack Web Development, MERN Stack, Data Science, Artificial Intelligence & Machine Learning, Cloud Computing, Cybersecurity, UI/UX Design, Android Development, .NET Technologies, and Advanced Web Development.

The institute regularly conducts **value-added courses, internships, workshops, seminars, faculty development programs, and research support activities** in collaboration with colleges and universities. Through Memorandum of Understanding (MoU) partnerships with academic institutions, we provide in-house training programs that help students gain practical knowledge and improve employability.

KICE Infosystems has successfully collaborated with several reputed institutions such as **PSGR Krishnammal College for Women, Sri Ramakrishna College for Women, AVP College of Arts and Science, St. Joseph's College for Women, and KPR College of Arts and Science** for technical training and academic projects.

With experienced trainers, structured course modules, and real-time project exposure, the Training and Research Division empowers students with strong technical skills and prepares them for successful careers in the IT industry.

1.2 OVERVIEW OF THE PROJECT

Product design is an important part of developing new products in today's competitive market. Designers usually start with ideas, sketches, and discussions before creating a final product. This process takes a lot of time and effort, especially during the early stages when many design ideas need to be explored and evaluated. Designers must consider product appearance, usability, material selection, and user needs before finalizing a design. Because of this, there is a growing need for intelligent tools that can help designers quickly visualize product concepts and experiment with multiple design layouts efficiently.

The rapid growth of Artificial Intelligence, Generative AI has become a powerful technology for creating new content such as text, images, and designs. Generative AI models learn patterns from large datasets and generate new outputs based on user input instructions. In product design, these models help convert conceptual ideas into visual representations automatically. Instead of manually drawing sketches or using complex modeling software, users can describe a product using natural language and obtain design visuals within minutes.

The project **"AI Product Design Generator with Generative AI"** uses this technology to assist users in creating product design concepts automatically. The system allows users to enter basic information such as product name, target audience, design style, material, color theme, and additional features. Based on these inputs, the system generates realistic product design images that represent the user's idea. The generated outputs follow structured design layouts such as centered composition, clean background, studio lighting, and product-focused visualization to simulate real product photography or industrial design concepts.

The system supports the creation of various product design layouts including minimalist designs, futuristic concepts, ergonomic products, luxury product visuals, and kid-friendly product styles. For example, users can generate designs such as a modern ergonomic office chair, a smart wearable device, a portable air purifier, a study lamp, kitchen accessories, or consumer electronics. The system ensures that the product remains the main focus while maintaining visual balance, proportion, and clarity in the design layout.

Different categories of products can be generated using this system, including **consumer electronics**, **home appliances**, **educational products**, **health and fitness devices**, **office accessories**, **kitchen products**, **children's products**, and **lifestyle gadgets**. This flexibility allows students, designers, and startups to explore multiple product domains without needing specialized design tools. The ability to generate multiple design variations also helps users compare alternative product appearances before selecting a preferred concept.

The application is developed using the Python Flask framework, which provides a lightweight and flexible platform for building web-based applications. A locally deployed Stable Diffusion model is used for image generation, allowing the system to run without depending on internet connectivity or paid APIs. Since all processing happens on the local machine, the application ensures data privacy, cost efficiency, and continuous availability, making it suitable for academic environments and offline demonstrations.

This project demonstrates how Generative AI can simplify and speed up the early stages of product design. It helps students, designers, and startups visualize ideas quickly and effectively while reducing manual effort. The system serves as a foundation for future AI-driven design assistants capable of generating intelligent layouts, detailed product specifications, and collaborative design workflows, thereby enhancing creativity, innovation, and modern product development practices.

Helping users transform ideas into visual designs The AI Product Design Generator using Generative AI is a web-based application developed to assist users in quickly transforming product ideas into visual design concepts. In traditional product development, designers often spend significant time creating sketches, experimenting with design variations, and refining concepts before arriving at a suitable product design. This process can be time-consuming and requires strong design skills and technical knowledge. The proposed system simplifies this process by automatically generating product design visuals based on simple user inputs.

The system works by allowing users to enter basic design details such as product name, target audience, design style, material type, color theme, and additional product features through a web interface. Based on these inputs, the system constructs a structured prompt and uses a locally deployed Stable Diffusion Generative AI model to generate realistic product design images. The generated images follow a clean and professional layout with studio lighting and product-focused visualization, making them useful for early-stage design exploration and concept validation.

This system is mainly used during the initial stage of product development, where multiple design ideas need to be explored quickly. Instead of manually creating several sketches or prototypes, users can generate different design variations within a short time. The system also allows users to download generated images and automatically creates a PDF design report containing the input details, prompts used, and generated design outputs. This makes the system useful for academic documentation, design presentations, and concept demonstrations.

The application is developed using Python and the Flask framework, with a frontend built using HTML, CSS, and JavaScript. The Stable Diffusion model runs locally using a .safe tensors file, allowing the system to operate offline without relying on paid APIs or internet connectivity. This ensures data privacy, cost efficiency, and continuous availability for users.

The primary users of this system include students, product designers, researchers, and startup innovators who need to visualize product ideas quickly. It is especially beneficial in educational environments where students can experiment with design concepts without needing advanced design software. By reducing manual effort and enabling rapid design generation, the system supports creativity, idea exploration, and modern AI-assisted product development.

Overall, the AI Product Design Generator provides an efficient platform for generating product design concepts using Generative AI technology, quickly and effectively.

OBJECTIVES OF THIS PROJECT

- To design and develop an AI-based product design generation system using Generative AI techniques.
- To enable users to create product design concepts by providing simple and structured design inputs.
- To generate realistic product design images using a locally deployed Stable Diffusion model.
- To reduce manual effort and time involved in early-stage product design and visualization.
- To provide an offline, cost-effective, and secure solution without dependency on paid APIs.
- To support academic and startup use by generating multiple design variations.

FEATURES OF PYTHON

- **Simple and Easy to Learn**

  Python has a simple and readable syntax, which makes it easy for beginners and developers to learn and understand the language quickly.

- **High-Level Programming Language**

  Python is a high-level language that allows programmers to write programs without worrying about complex memory management.

- **Platform Independent**

  Python programs can run on different operating systems such as Windows, Linux, and MacOS without making major changes to the code.

- **Open Source Language**

  Python is an open-source programming language, which means it is freely available for use, modification, and distribution.

- **Large Standard Library**

  Python provides a rich set of built-in libraries that support various functionalities such as file handling, web development, and data processing.

- **Supports Multiple Programming Paradigms**

  Python supports procedural programming, object-oriented programming (OOP), and functional programming, making it flexible for different types of applications.

- **Extensive Community Support**

  Python has a large developer community that continuously contributes libraries, frameworks, and tools to improve the language.

- **Integration Capability**

  Python can easily integrate with other programming languages such as C, C++, and Java, which helps in building complex applications.

1.3 SYSTEM SPECIFICATION

1.3.1 HARDWARE SPECIFICATION

Processor : Intel Core i5 / i7 (3.0 GHz or higher)

Hard Disk : 1 TB

RAM : 8 GB or above

Keyboard : Standard Keyboard

Mouse : Optical Mouse

1.3.2 SOFTWARE SPECIFICATION

Operating System : Windows 11 / Linux

Front End : Python (Tkinter / HTML, CSS)

Framework : Flask / Django

Database : MySQL / SQLite

1.3.2.1 LANGUAGE DESCRIPTION

INTRODUCTION TO FRONT END

PYTHON:

Python is a high-level, interpreted and general-purpose dynamic programming language that focuses on code readability. The syntax in Python helps the programmers to do coding in fewer steps as compared to Java or C. Python is widely used in bigger organizations because of its multiple programming paradigms. They usually involve imperative and object-oriented functional programming. It has a comprehensive and large standard library that has automatic memory management and dynamic features.

The software development companies prefer Python language because of its versatile features and fewer programming codes. Nearly 14% of the programmers use it on the operating systems like UNIX, Linux, Windows and Mac OS. Interactive

ADVANTAGES OR BENEFITS OF PYTHON

The Python language has diversified application in the software development companies such as in gaming, web frameworks and applications, language development, prototyping, graphic design applications, etc. This provides the language a higher plethora over other programming languages used in the industry. Some of its advantages are-

Extensive Support Libraries

It provides large standard libraries that include the areas like string operations, Internet, web service tools, operating system interfaces and protocols. Most of the highly used programming tasks are already scripted into it that limits the length of the codes to be written in Python.

Presence of Third Party Modules:

The Python Package Index (PyPI) contains numerous third-party modules that make Python capable of interacting with most of the other languages and platforms.

- **Integration Feature**

Python integrates the Enterprise Application Integration that makes it easy to develop Web services by invoking COM or COBRA components. It has powerful control capabilities as it calls directly through C, C++ or Java via Python. Python also processes XML and other markup languages as it can run on all modern operating systems through same byte code.

- **Improved Programmer's Productivity**

The language has extensive support libraries and clean object-oriented designs that increase two to ten fold of programmer's productivity while using the languages like Java, VB, Perl, C, C++ and C#.

- **Productivity**

With its strong process integration features, unit testing framework and enhanced control capabilities contribute towards the increased speed for most applications and productivity of applications. It is a great option for building scalable multi-protocol network applications.

- **Open Source and Community Development**

Python language is developed under an OSI-approved open source license, which makes it free to use and distribute, including for commercial purposes.

Further, its development is driven by the community which collaborates for its code through hosting conferences and mailing lists, and provides for its numerous modules.

- **Learning Ease and Support Available:**

Python offers excellent readability and uncluttered simple-to-learn syntax which helps beginners to utilize this programming language. The code style guidelines, PEP 8, provide a set of rules to facilitate the formatting of code. Additionally, the wide base of users and active developers has resulted in a rich internet resource bank to encourage development and the continued adoption of the language.

- **User-friendly Data Structures:**

Python has built-in list and dictionary data structures which can be used to construct fast runtime data structures. Further, Python also provides the option of dynamic high-level data typing which reduces the length of support code that is needed.

- **Simple and easy to Learn**

1. The syntaxes of python language are very simple. Anybody can remember the python language syntaxes, rules and regulations very easily.
2. The elegant syntaxes of the python language makes the people to learn python in easiest manner.
3. With out having any other programming languages knowledge, we can learn python directly.
4. The simple and powerful syntax of python language makes the programmers to express their business logic is less lines of code.

- **Platform Independent:**

1. Like Java programs, Python programs are also platform independent.
2. Once we write a Python program, it can run on any platform without rewriting once again.
3. Python uses PVM to convert python code to machine understandable code.

- **High Level Language :**

1. Python is a high level language.
2. While developing python applications, developers no need to bother about memory management.
3. Dynamically Typed Language :Python is a dynamically typed language.
4. I mean, in python there is no need to declare type of a variable.
5. Whenever we assign a value to the variable, based on the value the type will be allocated automatically.

- **Python supports POP & OOP:**

1. Python language supports both Procedural Oriented Programming and Object Oriented Programming features.
2. We can implement OOPs features like Encapsulation, polymorphic, inheritance and abstraction in python programming language.

- **Python is interpreted Language :**

1. Like PHP, Python is also interpreted language.
2. Python applications do not require explicit compilation so that compiler is not require for python software.
3. Python interpreter is responsible for execution of python applications.
4. When ever we run a python application, the python interpreter checks for the syntax errors.
5. If there is no syntax errors found in our code then the interpreter converts the code into intermediate code in the form of low level format and executes it.
6. The intermediate code of the python applications is known as byte code.
7. The extension of the byte code file is **.pye** (Python Compile Code).

- **Python is Embeddable :**

    1. We can embed the python code into other languages such as C,C++, Java and Etc..
    2. In order to provide the scripting capabilities to other languages we can use the python code in it.

- **Develop GUI & Web Application :**

    1. We can develop the GUI based applications using Python Languages.
    2. We can also develop the Web applications using Python Language.

Applications of Python:

- GUI based desktop applications
    - Image processing and graphic design applications
    - Scientific and computational applications
    - Games
- Web frameworks and web applications
- Enterprise and business applications
- Operating systems
- Language development
- Prototyping

TOOLS AND TECHNOLOGY USED

FLASK:

The Flask framework is actually a glue, a very nice one, that sticks together the amazing Werkzeug and Jinja2 frameworks, responsible for answering requests and presenting the output (HTML, maybe). In the MVC architecture, also known as Model-View-Controller, Flask covers C and V. But where is M? Flask does not provide you with an integrated model layer out-of-the-box as that is not actually needed for a web application. If you do need to work with a database, just pick your database solution from the many available and create your own model layer, which is not hard, and be happy! The concept of a micro-framework, with good intentions and made just for Flask, is all about giving you the the smallest (but also the most useful) feature set you need, and one that won't get in the way.

Just what are the features that must be in the framework

- A development server and debugger
- Unicode support
- WSGI compliance
- A unit-test client
- URL routing
- Request dispatching
- Secure cookies
- Sessions
- Jinja2 templates

Why is Flask a good web framework choice?

Flask is considered more Pythonic than the Django web framework because in common situations the equivalent Flask web application is more explicit. Flask is also easy to get started with as a beginner because there is little boilerplate code for getting a simple app up and running.

How does Flask relate to the Pallets Projects?

Flask was originally designed and developed by Armin Ronacher as an April Fool's Day joke in 2010. Despite the origin as a joke, the Flask framework became wildly popular as an alternative to Django projects with their monolithic structure and dependencies.

Flask's success created a lot of additional work in issue tickets and pull requests. Armin eventually created The Pallets Projects collection of open source code libraries after he had been managing Flask under his own GitHub account for several years. The Pallets Project now serves as the community-driven organization that handles Flask and other related Python libraries such as Lektor, Jinja and several others.

2.SYSTEM STUDY

2.1. EXISTING SYSTEM

In the existing system, product design is mostly done using manual methods such as hand sketches, paper drawings, or basic digital drawing tools. Designers usually depend on their personal creativity and experience to visualize a product idea. To create realistic designs, advanced software like CAD tools or 3D modeling applications are used, which require strong technical skills and a long learning process. This makes the initial design phase slow and effort-intensive.

In many cases, designers and students need to spend a lot of time brainstorming, drawing multiple drafts, and revising designs before reaching a suitable concept. If changes are required, the entire design process must be repeated. This traditional approach is not efficient for quickly generating multiple design ideas and is difficult for beginners who lack professional design skills.

In the existing system, product design is mainly carried out using traditional methods such as manual sketches, paper drawings, and basic digital design tools. Designers usually depend on their creativity and experience to visualize product concepts.

To create detailed product designs, professional software such as Computer-Aided Design (CAD) tools or 3D modeling applications are often used. These tools help designers create digital product models and visualizations, but they require strong technical skills and specialized training.

Before the introduction of AI-based design systems, the design process was implemented manually. Designers first gathered ideas and requirements, then created rough sketches of the product. After several revisions and improvements, the design was converted into a digital model using design software. If changes were needed, the designer had to manually edit the design or recreate parts of the model. This process required significant time, effort, and technical knowledge.

The existing system also relied heavily on human effort to generate multiple design variations. Designers needed to manually modify product shapes, materials, colors, and layouts to explore different design possibilities. For beginners or students who lack professional design experience, visualizing product concepts using advanced software can be difficult.

2.1.1. DRAWBACKS OF EXISTING SYSTEM

1. Product design process is time-consuming and requires manual effort.
2. Advanced design tools need high technical knowledge and training.
3. Creating multiple design variations takes a lot of time.
4. Beginners and students find it difficult to visualize product ideas.
5. Manual design methods slow down early product development stages.
6. Design inspiration depends heavily on individual creativity and experience.
7. In the existing system, product design is mainly carried out using traditional methods such as manual sketches, paper drawings, and basic digital design tools.
8. Designers usually depend on their creativity and experience to visualize product concepts.
9. To create detailed product designs, professional software such as Computer-Aided Design (CAD) tools or 3D modeling applications are often used.
10. These tools help designers create digital product models and visualizations, but they require strong technical skills and specialized training.
11. Before the introduction of AI-based design systems, the design process was implemented manually.
12. Designers first gathered ideas and requirements, then created rough sketches of the product. After several revisions and improvements, the design was converted into a digital model using design software.
13. If changes were needed, the designer had to manually edit the design or recreate parts of the model.
14. This process required significant time, effort, and technical knowledge.
15. The existing system also relied heavily on human effort to generate multiple design variations.
16. Designers needed to manually modify product shapes, materials, colors, and layouts to explore different design possibilities.
17. For beginners or students who lack professional design experience, visualizing product concepts using advanced software can be difficult.
18. Therefore, the traditional product design process is slow and requires skilled professionals.

2.2. PROPOSED SYSTEM

The proposed system introduces an AI-based product design generator that automatically creates product design images using Generative AI techniques. Users only need to provide simple inputs such as product name, target audience, design style, and additional features. Based on these inputs, the system generates realistic product design visuals using a locally deployed Stable Diffusion model. The application is developed using the Python Flask framework and runs completely on a local system. Since the model works offline, there is no dependency on paid APIs or internet connectivity. The system also supports generating multiple design variations and automatically creates a downloadable PDF design report, making it suitable for academic projects and design documentation. The proposed system introduces an AI Product Design Generator using Generative AI that automatically creates product design images based on simple user inputs. The system allows users to enter details such as product name, target audience, design style, material, color theme, and additional features through a user-friendly web interface.

After receiving the input, the system generates a structured prompt and uses a locally deployed Stable Diffusion Generative AI model to create realistic product design images. The generated images follow a professional layout with studio lighting, centered composition, and clean backgrounds to represent product concepts clearly. The application is developed using the Python programming language and Flask web framework, while the frontend interface is created using HTML, CSS, and JavaScript. The system runs entirely on a local machine using a safetensors Stable Diffusion model, which allows it to work without internet connectivity or paid APIs.Another important feature of the proposed system is the ability to generate multiple design variations for the same product concept. This helps users compare different visual designs quickly. In addition, the system automatically generates a PDF design report that includes user inputs, the generated prompt, and the output images. This report can be used for academic submissions, presentations, or design documentation. Overall, the proposed system simplifies the product design process by reducing manual effort and enabling users to generate product design concepts quickly using AI technology.

2.2.1 ADVANTAGES OF PROPOSED SYSTEM

1. Automatically generates product design images with minimal user input.
2. Reduces time and effort required for early-stage product design.
3. Works offline and does not depend on paid third-party APIs.
4. Easy to use for students, beginners, and non-designers.
5. Supports multiple design variations for better comparison.
6. Provides downloadable design reports for documentation and presentation.
7. The system provides an easy-to-use interface suitable for students, designers, and beginners.
8. Since it uses open-source tools and models, it reduces software licensing costs.
9. The system generates downloadable PDF design reports for project submissions and presentations.

3.SYSTEM DESIGN AND DEVOLOPMENT

3.1 INPUT DESIGN

The input design of the system is implemented through a structured and user-friendly web interface developed using HTML, CSS, and Flask templates in the index.html page. The user interface is designed with a clean layout where all product design parameters are arranged logically using labeled input fields and grouped sections. The form collects essential inputs such as Product Name, Target Audience, Design Style, Render Type, Material, Color Theme, Extra Features, Quality Level, Seed value, and Variations count (1 or 3). Dropdown menus are used for predefined options like design style and quality level to reduce user confusion, while text boxes allow flexible input for product-specific details. The UI also includes placeholders and clear labels to guide users in entering meaningful design information.

From a usability perspective, the interface applies HTML validation techniques such as the required attribute for mandatory fields and appropriate input types like text, number, and select to minimize incorrect data entry. The optional seed field allows users to either generate random designs or reproduce the same design output when needed. The layout is responsive and centered, ensuring easy interaction even for users without technical knowledge. This UI design ensures that users can quickly configure product parameters and submit requests without complexity.

On the backend side, Flask receives form data using request.form.get() and stores the values in a structured dictionary (form_data). The system removes unnecessary spaces, safely handles optional inputs, and converts numeric values such as seed and variation count into integer format. Based on the selected quality level, the application automatically adjusts inference steps to balance generation speed and image quality, which is especially important for CPU-based execution. Thus, the input design combines an intuitive UI with reliable backend processing to produce stable AI prompts. Input design is an important part of the system design process. It focuses on how the input data is collected from users and provided to the system for further processing. In the Diabetic Retinopathy Detection system, the input design is developed to ensure that medical images and related details are captured correctly and efficiently.

The system uses a web-based interface where users upload retinal images and provide necessary details. The input design ensures that the data entered by the user is accurate, valid,

and properly formatted before it is processed by the prediction model. The main objective of input design is to reduce errors during data entry and provide reliable input for the detection system. The input process of the system can be divided into the following stages. \

This module displays the final prediction result generated by the trained machine learning model. After the retinal image is uploaded and processed using Python libraries, the system predicts whether the image shows signs of diabetic retinopathy or a normal condition. The result is displayed on the webpage in a clear text format such as “Diabetic Retinopathy Detected” or “No Diabetic Retinopathy Detected.” This helps users quickly understand the diagnostic outcome.

Uploaded Image Visualiz

The system also displays the uploaded retinal image on the output page. This allows the user to verify that the correct image has been processed by the system. The Flask application retrieves the uploaded image and renders it in the output template so the user can visually confirm the input image along with the prediction result.

Confidence Score Output

The trained model calculates a probability score for the prediction. This confidence score indicates how strongly the model predicts the presence or absence of diabetic retinopathy. The score is displayed as a percentage or decimal value, helping users understand the reliability of the prediction.

Result Message Notification

The system generates a clear message to inform the user about the analysis result. If diabetic retinopathy is detected, the system may display a warning message suggesting medical consultation. If no disease is detected, the system shows a normal status message. This output message improves user understanding and makes the system more interactive.

Web Page Output Interface

The final output is presented through a web interface developed using Flask and HTML templates. The processed results from the Python backend are passed to the front-end template and displayed in an organized format. The interface includes the uploaded image, prediction result, and status message, ensuring that users can easily interpret the output generated by the system.

3.2 OUTPUT DESIGN

The output design focuses on presenting generated product design results through a visually clear and professional user interface. After image generation, the Flask application dynamically updates the same webpage to display results inside an **output section**. The UI includes a structured result panel where generated images appear neatly aligned with proper spacing and headings. Depending on the selected variation count, the interface displays either a single design image or a gallery view showing three design variations side-by-side, allowing users to compare different visual concepts easily.

The generated images are stored in the static/generated/ directory and displayed using URL-safe paths to ensure browser compatibility. Each image is rendered with proper scaling and styling so that the product remains clearly visible. The output interface also includes download buttons, enabling users to save generated images directly for presentations, reports, or further design refinement. Visual consistency is maintained through CSS styling such as borders, shadows, and centered alignment to create a professional appearance suitable for demonstrations.

In addition to visual outputs, the system automatically generates a **PDF design report** stored in static/reports/latest_report.pdf. The UI provides a dedicated download option through a Flask route (/download_report), allowing users to obtain a formatted report containing input details, generated prompts, and embedded images. The report layout follows an academic format, making it suitable for project submission. Overall, the output design integrates backend processing with an interactive UI to ensure that generated results are clearly displayed, easily accessible, and practically usable.

Prediction Result Display

This module displays the final prediction result generated by the trained machine learning model. After the retinal image is uploaded and processed using Python libraries, the system predicts whether the image shows signs of diabetic retinopathy or a normal condition. The result is displayed on the webpage in a clear text format such as “Diabetic Retinopathy Detected” or “No Diabetic Retinopathy Detected.” This helps users quickly understand the diagnostic outcome.

Uploaded Image Visualization

The system also displays the uploaded retinal image on the output page. This allows the user to verify that the correct image has been processed by the system. The Flask application retrieves the uploaded image and renders it in the output template so the user can visually confirm the input image along with the prediction result.

Confidence Score Output

The trained model calculates a probability score for the prediction. This confidence score indicates how strongly the model predicts the presence or absence of diabetic retinopathy. The score is displayed as a percentage or decimal value, helping users understand the reliability of the prediction.

Result Message Notification

The system generates a clear message to inform the user about the analysis result. If diabetic retinopathy is detected, the system may display a warning message suggesting medical consultation. If no disease is detected, the system shows a normal status message. This output message improves user understanding and makes the system more interactive.

Web Page Output Interface

The final output is presented through a web interface developed using Flask and HTML templates. The processed results from the Python backend are passed to the front-end template and displayed in an organized format. The interface includes the uploaded image, prediction result, and status message, ensuring that users can easily interpret the output generated by the system.

3.3 DESCRIPTION OF MODULES

1. User Interface Module

This module handles the interaction between the user and the system through a web-based interface developed using Flask and HTML forms. The user provides structured product design inputs such as product name, target audience, design style, render type, material, color theme, extra features, quality level, seed value, and number of design variations. These inputs are collected through a POST request and sent to the Flask backend for processing.

When the form is submitted, Flask captures input values using the request object and stores them in a dictionary structure for further processing. Input validation ensures required fields are filled and optional values are handled safely. This module acts as the entry point of the application workflow.

```
form_data["product_name"] = request.form.get("product_name", "")
form_data["audience"] = request.form.get("audience", "")
form_data["style"] = request.form.get("style", "Minimalist")
```

2. Prompt Generation Module

This module converts structured user inputs into an optimized natural-language prompt suitable for the generative AI model. Instead of directly using user text, the system constructs a formatted prompt template that includes product constraints such as centered composition, studio lighting, and clean background. A negative prompt is also applied to prevent unwanted elements like text, logos, or multiple objects.

The prompt builder ensures consistency and improves generation accuracy by combining multiple input attributes into a single descriptive instruction. This improves model guidance during image generation.

```
prompt = (
    f"A single {product_name} designed for {audience}. "
    f"{style} style, studio product photo, "
    f"center composition, plain white background."
)
```

Negative prompts guide the model away from undesired visual artifacts.

```
negative_prompt = "text, watermark, logo, blurry, extra objects"
```

3. Generative AI Model Module

This module performs product image generation using a locally deployed Stable Diffusion model stored in .safe tensors format. The model is loaded once during application startup along with CLIP tokenizer and text encoder components. Running the model locally allows offline execution without API dependency.

During generation, the prompt is converted into embeddings using CLIP encoding. The diffusion process gradually removes noise over multiple inference steps to produce a final image. CPU optimization techniques such as attention slicing are enabled to reduce memory usage.

```
result = pipe(
    prompt=prompt,
    negative_prompt=negative_prompt,
    num_inference_steps=steps,
    guidance_scale=7.5
)
image = result.images[0]
```

4. Image Generation and Variation Module

This module manages multiple design outputs and file storage operations. Based on the selected variation count, the system generates one or three images by adjusting the random seed value. This produces visually different design interpretations while maintaining the same concept.

Each generated image is saved dynamically inside the static/generated/ directory using unique filenames created with UUID to avoid overwriting existing files.

```
filename = f"{uuid.uuid4().hex}.png"
image.save(os.path.join(OUTPUT_DIR, filename))
```

5. Design Report Generation Module

This module automatically creates a professional PDF design report containing user inputs, generated prompts, and output images. The ReportLab library is used to generate A4-format documents suitable for academic submission and presentations.

After image generation, the system embeds images and text content into the PDF layout and saves it inside the static/reports/ folder.

```
c = canvas.Canvas("latest_report.pdf", pagesize=A4)
c.drawString(40, height-50, "AI Product Design Report")
c.drawImage(img_path, 40, 300, width=400, height=250)
c.save()
```

6. Result Display

This module presents generated outputs to the user through the web interface. The system dynamically displays either a single image or a gallery view when multiple variations are generated. Users can download images or the generated PDF report directly from the result page.

Flask renders the output using template rendering, passing generated image paths and prompt details to the HTML page.

```
return render_template(
   "index.html",
   images=images,
   prompt_used=prompt_used
)
```

Algorithms and Modeling

1) Prompt Construction Algorithm

The system converts structured user inputs (product name, audience, style, render type, material, color theme, features) into a single optimized text prompt. A prompt template is used to keep the output product-focused by adding constraints such as “single object”, “center composition”, “plain white background”, and “studio lighting”. This improves the relevance of generated images and reduces random objects in the output.

Along with the main prompt, the system applies a **negative prompt** to avoid unwanted elements like text, watermark, logo, people, messy background, blur, and low quality. This works like a rule-based filter that guides the generative model away from common errors. The prompt-building logic is deterministic, so the same inputs produce the same prompt every time.

2) Image Generation Model (Stable Diffusion)

The core model used is **Stable Diffusion**, which is a **latent diffusion model**. Instead of generating an image directly in pixel space, the model generates an image in a compressed latent space using a **Variational Autoencoder (VAE)**. This makes generation faster and less memory heavy compared to pixel-space diffusion.

Stable Diffusion uses a **U-Net denoising network** trained to remove noise step-by-step from a latent representation. The user prompt is converted into embeddings using **CLIP tokenizer + CLIP text encoder**, and these embeddings condition the U-Net to guide the generation toward the requested product concept. After denoising, the VAE decoder converts the final latent into the output image.

3) Diffusion Sampling Algorithm (Inference Steps)

During generation, the model starts with random noise and gradually refines it into a meaningful image over multiple iterations called **inference steps**. In this project, inference steps are chosen based on user-selected quality: Low/Medium/High. A higher step count generally produces better details but increases generation time, which is important because the model runs on CPU.

The system also uses **Classifier-Free Guidance (CFG)** controlled by a parameter called guidance_scale. A moderate guidance scale improves how strongly the model follows the prompt. If the guidance is too low, results may be generic; if too high, results may become distorted. The project uses a balanced default value suitable for product-style images.

4) Variation Generation Algorithm (Multiple Outputs)

To produce multiple design ideas for the same product, the system supports generating **three variations**. This is done using seed management: if the user gives a seed, the system generates variation images by incrementing it (seed, seed+1, seed+2). If no seed is given, the model uses random seeds to create different outputs.

This approach helps users compare alternative looks such as different shapes, colors, or styles while keeping the same design intent. It is useful for brainstorming and early-stage product ideation, where multiple concepts are needed before selecting a final design.

5) PDF Report Generation Algorithm

After image generation, the system automatically creates a **PDF design report** using ReportLab. The report includes the user inputs, the final prompt used, and the generated output images. Images are resized to fit into standard A4 pages while preserving aspect ratio, and additional pages are created when multiple variations are generated.

This report generation step converts the AI output into a reusable design document for academic submission, presentations, and project demonstration. It acts as an automated documentation module that completes the design workflow from input → generation → report.

4.TESTING AND IMPLEMENTATION

4.1 TESTING

System testing is the process of exercising software with the intent of finding and ultimately correcting errors. This fundamental philosophy does not change for web applications, because Web-based systems and application reside on a network and interoperate with many different operating system, browsers, hardware platforms, and communication protocols; the search for errors represents a significant challenge for web application.

The distributed nature of client\server environments, the performance issues associated with transaction processing, the potential presence of a number of different hardware platforms, the complexities of network communication, the need to serve multiple clients from a centralized database and the requirements imposed on the server all combine to make testing of client\server architectures.

System testing is actually a series of different tests whose primary purpose is to fully exercise the computer based system. System testing is the state of implementation that is aimed at assuring that the system works accurately and efficiently. Testing is the vital to the success of the system. System testing makes the logical assumption that if all the parts of the system are correct, the goal will be successfully achieved.

THE OBJECTIVE OF TESTING IS AS FOLLOWS:

- Testing is the process of executing a program with the intent of finding an error
- A successful test is that one of the cover of undiscovered error

TESTING ISSUES

- Client GUI considerations
- Target environment and platform diversity considerations
- Distributed database considerations
- Distributed processing considerations

4.1.1 TESTING METHODOLOGIES

System testing is the state of implementation, which is aimed at ensuring that the system works accurately and efficiently as expect before live operation commences. It certifies that the whole set of programs hang together. System testing requires a test plan that consists of several key activities and steps for run program, string, system and user acceptance testing. The implementation of newly designed package is important in adopting a successful new system.

Testing is an important stage in software development. The system test in implementation stage in software development. The system test in implementation should be confirmation that all is correct and an opportunity to show the users that the system works as expected. It accounts the largest percentage of technical effort in the software development process.

Testing phase in the development cycle validates the code against the functional specification. Testing is vital to the achievement of the system goals. The objective of idations and system tests were planned and executed.

4.2 TYPES OF TESTING

4.2.1.Unit Testing

Testing is to discover errors. To fulfill this objective a series of test step unit, integration, Unit testing is done to check each small function or module of the project separately to ensure it works correctly. In this project, unit tests can be performed for functions like prompt generation, quality-to-steps conversion, seed handling, and file path formatting. For example, when the user selects “High” quality, the function should return the correct number of inference steps, and when a seed is given, the system should accept it without errors.

Unit testing also verifies that individual components produce expected outputs before connecting them with other modules. For instance, the image-saving function should always store the generated image inside the correct static/generated/ folder and return a URL-friendly

path using forward slashes. By testing each function separately, we reduce bugs and ensure the backend code is stable.

The User Authentication Module was tested for login validation, password encryption using hashing mechanisms, invalid login detection, session management, logout functionality, and account lock mechanism after multiple failed attempts.

The Database Module was verified for accurate insertion, update, and retrieval of prediction records, enforcement of primary and foreign key constraints, prevention of duplicate entries, and handling of database connection failures.

All individual modules performed as expected without logical or runtime errors, confirming that each unit was stable, reliable, and ready for integration.

4.2.2. Integration Testing

Integration testing is used to verify that different modules of the application work properly when combined together. In this project, integration testing ensures that the UI form inputs are correctly passed to the Flask backend, and that the backend successfully generates a prompt, calls the Stable Diffusion pipeline, saves the image, and returns the result back to the output page.

This testing also checks the connection between image generation and report generation modules. For example, after generating one or three images, the system should automatically create a PDF design report including the prompt and generated images. Integration testing confirms that the entire workflow is connected correctly and no module fails when interacting with another module

4.2.3.Validation Testing

Validation testing is done to confirm that the system produces correct results according to user expectations and project objectives. In this project, validation testing ensures that the generated design images match the given user inputs such as product name, style, and audience in a reasonable way. For example, if the user enters “Water Bottle – Kids – Minimalist,” the output should look like a bottle design with a clean, simple appearance suitable for children.

Validation testing also checks whether the project meets the purpose stated in the abstract and objectives, such as reducing manual design effort and providing multiple design variations. It confirms that the system is useful, user-friendly, and produces outputs that can be used as early-stage product design references. This testing gives confidence that the project is valid and ready for academic submission.

4.2.4.System Testing

System testing evaluates the complete application as a full system to ensure it meets the required functionality. In this project, system testing is performed by running the application on a laptop and checking the full flow: opening the website, entering input values, generating images, displaying the output correctly, downloading the image, and generating a PDF report successfully.

System testing also checks performance and usability in real conditions. Since the model runs on CPU, the system test includes verifying that the loading indicator appears during processing, the app does not crash during long generation time, and the output image is still displayed correctly after generation. This confirms that the complete system is reliable for demo and academic use.

4.3.SYSTEM IMPLEMENTATION AND MAINTENANCE

System Implementation

The system implementation of the AI Product Design Generator is developed as a web application using the Python Flask framework. The frontend is implemented with HTML/CSS/JavaScript to collect structured inputs such as product name, target audience, design style, render type, material, color theme, quality level, seed, and number of variations. When the user submits the form, Flask handles the HTTP POST request, validates and parses the inputs, and constructs an optimized prompt and negative prompt using a template-based prompt generation function. The application then invokes a locally loaded Stable Diffusion pipeline (from a .safetensors model file) along with locally stored CLIP tokenizer and CLIP text encoder. The model is initialized once at application startup to avoid repeated loading overhead, and CPU memory optimization is enabled using attention slicing to ensure smooth execution on systems without GPU.

During inference, the system performs diffusion sampling based on selected quality settings by controlling parameters such as inference steps and guidance scale. The generated image outputs are saved dynamically into the static/generated/ folder with unique filenames, and URL-safe paths are returned to display the results on the output page. If the user selects multiple variations, the system generates multiple images by controlling randomness through seed management (seed, seed+1, seed+2) to produce different design options while keeping the same concept. After image generation, the application automatically creates a professional PDF design report using the ReportLab library, which includes user inputs, the final prompt used, and the generated images embedded in A4 format. The report is stored in static/reports/ and is made available through a Flask route for downloading, completing the full workflow from user input → AI generation → result display → documentation

System Maintenance

System maintenance is an ongoing process performed after deployment to ensure the continuous and efficient operation of the system. It includes monitoring system performance, correcting errors, updating features, and improving security. Maintenance is essential because software systems may require modifications due to changing user requirements, technological advancements, or newly identified issues.

There are three main types of maintenance involved in this project:

1. Corrective Maintenance

Corrective maintenance involves fixing errors or bugs that are discovered after deployment. This may include resolving prediction inaccuracies, fixing database connection issues, correcting report generation errors, or handling unexpected runtime exceptions.

2. Perfective Maintenance

Perfective maintenance focuses on enhancing system performance and adding new features. For example, upgrading from binary classification to multi-class DR stage prediction, improving model accuracy, enhancing the user interface, or optimizing database performance are considered perfective improvements.

3. Preventive Maintenance

Preventive maintenance aims to reduce future system failures. This includes updating libraries, improving code structure, performing regular database backups, strengthening security mechanisms, and optimizing model performance to avoid degradation over time.

Regular monitoring, log analysis, performance tuning, and security updates ensure that the system remains reliable, secure, and efficient. Proper maintenance increases system lifespan and ensures that it continues to support medical professionals effectively.

5.CONCLUSION

The project **“AI Product Design Generator with Generative AI”** successfully demonstrates how artificial intelligence can be used to simplify and improve the product design process. By allowing users to enter basic design details and automatically generating product design images, the system reduces the time and effort required in the early stages of product development. The use of a locally deployed Stable Diffusion model ensures that the application works offline, remains cost-effective, and protects user data.

This project proves that Generative AI can be effectively applied in academic and real-world design scenarios. It helps students, designers, and startups visualize product ideas quickly without requiring advanced design skills. Overall, the system provides a strong foundation for future AI-based design tools and highlights the potential of Generative AI in supporting creativity, innovation, and efficient product development.

The project **“AI Product Design Generator with Generative AI”** demonstrates how modern artificial intelligence technologies can assist in the early stages of product design and innovation. In traditional product development processes, designers and developers spend significant time creating sketches, discussing ideas, and visualizing different design possibilities. This process often requires specialized tools and professional design skills. The developed system simplifies this process by allowing users to generate product design concepts automatically using simple textual inputs. By combining the capabilities of Generative AI with a user-friendly web interface, the system provides a practical solution for quickly visualizing product ideas.

The application is developed using the **Python programming language and Flask web framework**, which together provide a flexible and lightweight environment for building AI-powered web applications. The integration of the **Stable Diffusion generative model** enables the system to transform textual descriptions into visual product design images. Users can provide details such as product name, target audience, design style, materials, color theme, and additional features, and the system generates corresponding design outputs. This approach reduces manual effort and helps users explore multiple design possibilities within a short period of time.

One of the major advantages of the system is its **offline functionality**, where the AI model runs locally on the user's system without requiring internet connectivity or paid API services. This ensures data privacy and makes the application suitable for academic

institutions and research environments. The system also supports generating multiple design variations and provides a downloadable **PDF design report**, which includes the user inputs, generated prompts, and output images. These features make the system useful not only for idea generation but also for documentation and presentation purposes.

Overall, the project highlights the potential of **Generative AI in supporting creative workflows and accelerating product innovation**. It shows how artificial intelligence can assist designers, students, and startup teams in visualizing product ideas more efficiently. The system serves as a foundation for future AI-assisted design platforms that can integrate advanced features such as 3D modeling, interactive design customization, and collaborative design environments. By bridging the gap between conceptual ideas and visual design outputs, this project demonstrates the growing importance of AI technologies in modern product development and digital creativity.

5.2.FUTURE ENHANCEMENTS

In the future, this project can be enhanced by integrating advanced AI models to generate detailed product descriptions, specifications, and feature lists automatically. By connecting the system with text-based generative AI models, users can receive complete product documentation along with design images. Database integration can also be added to store design history, user inputs, and generated reports, allowing users to revisit and compare previous designs easily.

Another important enhancement is the addition of real-time collaboration and customization features. Users could modify generated designs by adjusting parameters such as size, color, or material interactively. The system can also be extended to support GPU acceleration for faster image generation and mobile-friendly interfaces for wider accessibility. These improvements would make the application more powerful, user-centric, and suitable for commercial product design environments.

The current system provides an efficient platform for generating product design concepts using Generative AI. However, several improvements can be implemented in future versions to enhance the system's capabilities and usability. One possible enhancement is the integration of **more advanced generative models** that can produce higher quality and more detailed product images. As AI technology continues to evolve, newer models with improved image resolution and design accuracy can be incorporated into the system to generate more realistic product designs. This would allow users to obtain design outputs that are closer to real-world product prototypes.

Another important enhancement would be the addition of **3D product design generation**. Currently, the system generates two-dimensional product images based on user prompts. In the future, the system could integrate AI models capable of generating **3D product models**, allowing users to view designs from multiple angles. This feature would be particularly useful for product engineers and industrial designers who need a better understanding of product structure and dimensions before manufacturing.

The system can also be improved by introducing **interactive design customization tools**. Users could be given options to modify generated designs by adjusting parameters such as shape, color variations, material textures, and product dimensions. This interactive design feature would allow users to refine AI-generated concepts and make modifications according to their requirements, leading to more personalized product designs.

Another possible enhancement is the integration of **database storage and project management features**. Currently, the generated images and reports are stored locally. In future versions, the system could include user accounts where designers can save their projects, revisit previously generated designs, and manage multiple design sessions. This would make the application more suitable for professional design workflows and collaborative environments.

The system could also support **multi-model AI integration**, where different AI models are used for different tasks such as product description generation, image creation, and design recommendation. For example, one AI model could generate detailed product specifications while another model generates visual designs. Combining multiple AI technologies would significantly improve the overall intelligence and functionality of the system.

Finally, the application can be expanded into a **cloud-based platform** that allows remote access and collaboration. Designers from different locations could work together, share design concepts, and provide feedback in real time. Additional features such as automated design evaluation, market trend analysis, and AI-based product recommendation could also be included. These enhancements would transform the system into a comprehensive AI-powered product development platform capable of supporting modern digital design workflows.

BIBLIOGRAPHY

- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville.
  Deep Learning. MIT Press, 2016.
  (Fundamental concepts of neural networks, generative models, and deep learning.)
- Géron, Aurélien.
  Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media, 2nd Edition, 2019.
  (Practical implementation of ML and deep learning models.)
- Chollet, François.
  Deep Learning with Python. Manning Publications, 2018.
  (Neural networks and model implementation using Python.)
- McKinney, Wes.
  Python for Data Analysis. O'Reilly Media, 2017.
  (Data handling and processing in Python.)
- Grinberg, Miguel.
  Flask Web Development: Developing Web Applications with Python. O'Reilly Media, 2nd Edition, 2018.
  (Comprehensive guide to Flask framework.)
- Nikhil Buduma and Nicholas Locascio.
  Fundamentals of Deep Learning. O'Reilly Media, 2017.
  (Core deep learning principles and architectures.)

RESEARCH PAPERS

Robin Rombach, Andreas Blattmann, Dominik Lorenz, et al.

"High-Resolution Image Synthesis with Latent Diffusion Models."

Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022.

(Foundation paper for Stable Diffusion.)

Alec Radford, et al.

"Learning Transferable Visual Models From Natural Language Supervision."

International Conference on Machine Learning (ICML), 2021.

(CLIP model used for text-image alignment.)

Diederik P. Kingma and Max Welling.

"Auto-Encoding Variational Bayes."

International Conference on Learning Representations (ICLR), 2014.

(Variational Autoencoders – core component of latent diffusion.)

Jonathan Ho, et al.

"Denoising Diffusion Probabilistic Models."

NeurIPS, 2020.

(Foundation of diffusion-based image generation.)

Journal of Artificial Intelligence Research (JAIR)

(Articles related to generative AI and machine learning.)

IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)

(Research on computer vision and image generation.)

WEBSITES

•**Python Software Foundation**
https://www.python.org
(Official Python documentation)

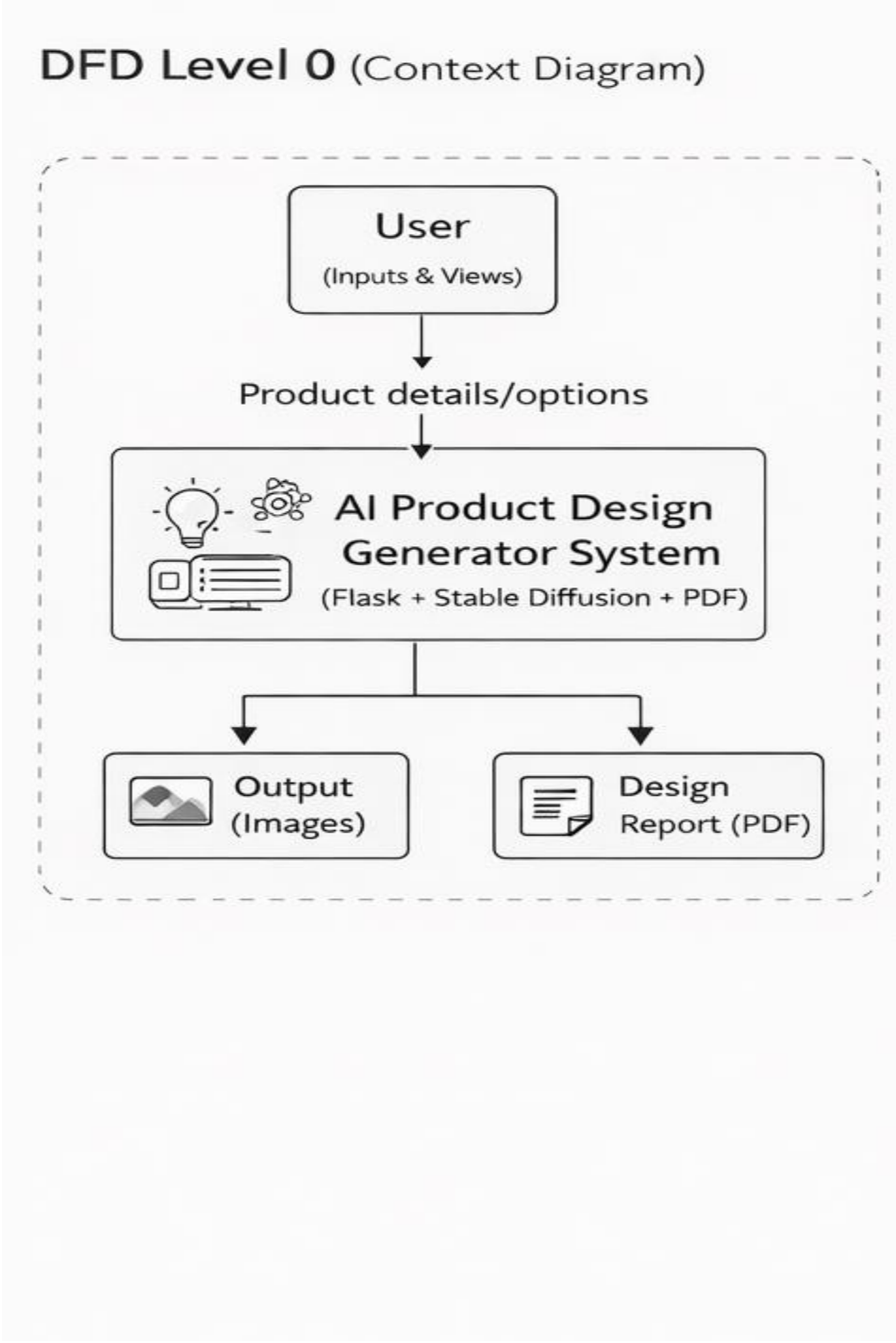
•**Pallets Projects** – Flask Documentation
https://flask.palletsprojects.com

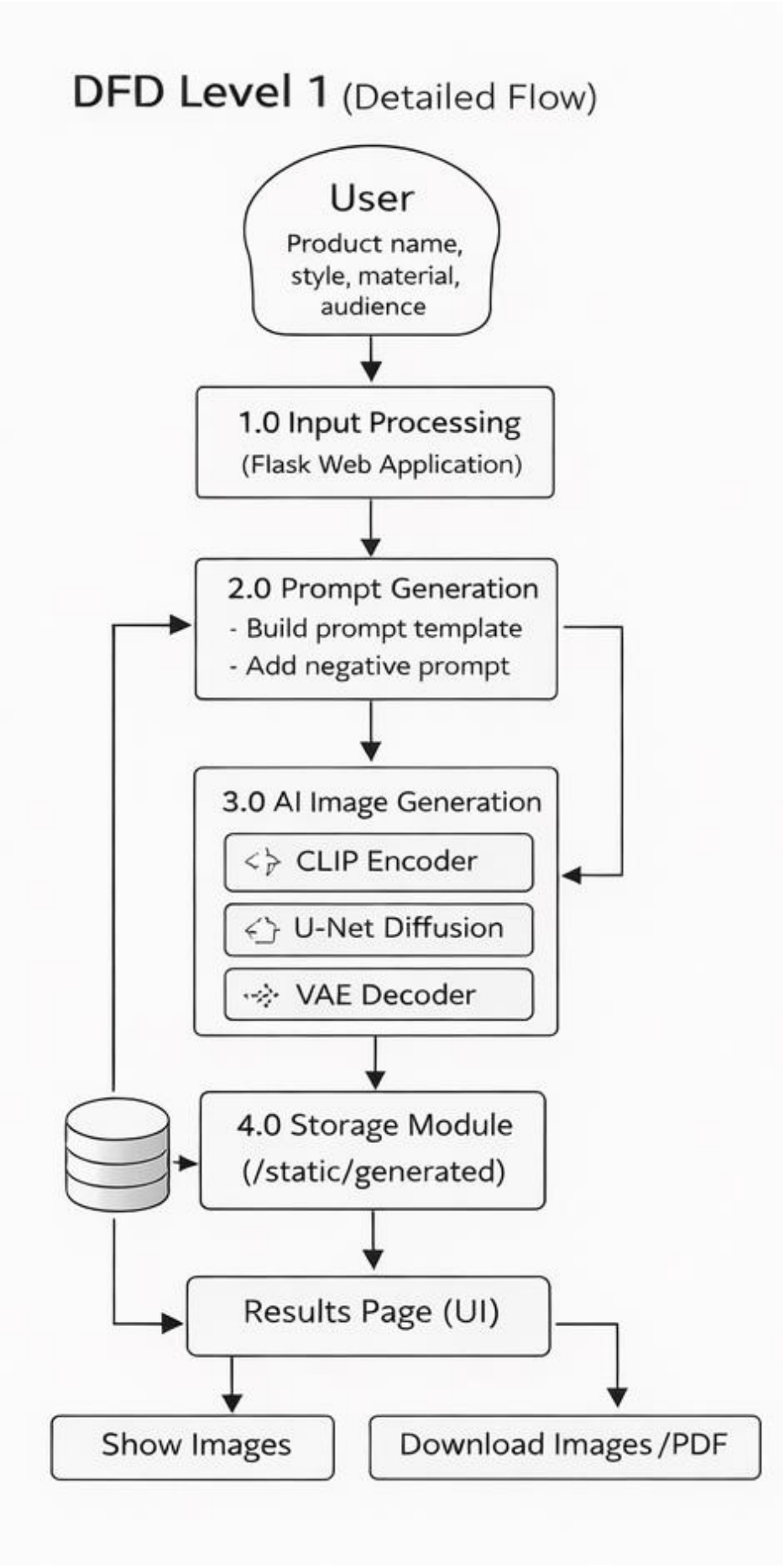
•**Hugging Face**
https://huggingface.co
(Diffusers library and Stable Diffusion implementation)

•**Stability AI**
https://stability.ai
(Stable Diffusion model information)

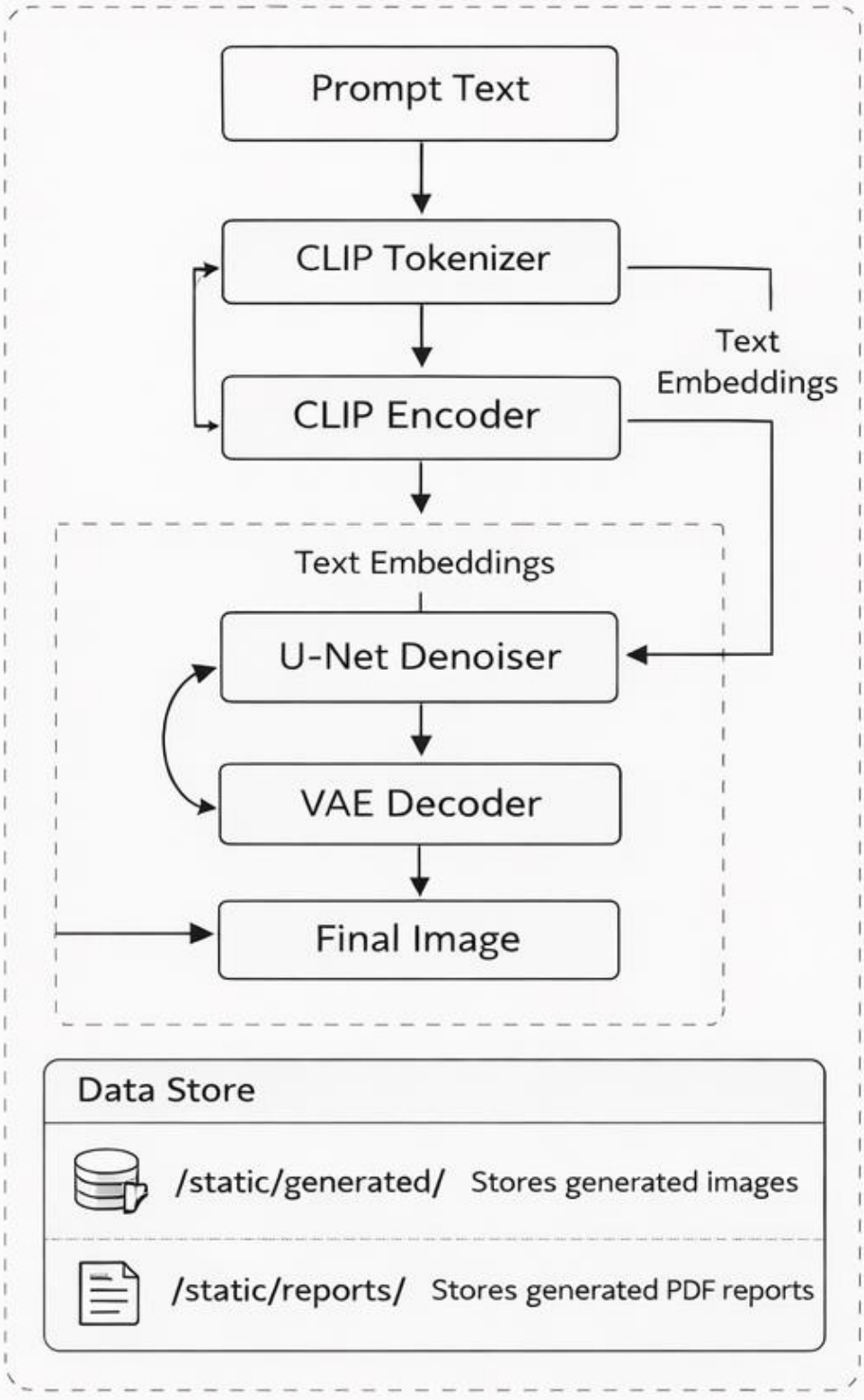
•**PyTorch**
https://pytorch.org
(Deep learning framework used in diffusion models)

SYSTEM ARCHITECTURE DIAGRAM




DFD Level 1 (Detailed Flow)
User
Product name, style, material, audience
1.0 Input Processing
(Flask Web Application)
2.0 Prompt Generation
- Build prompt template
- Add negative prompt
3.0 AI Image Generation
CLIP Encoder
U-Net Diffusion
VAE Decoder
4.0 Storage Module
(/static/generated)
Results Page (UI)
Show Images
Download Images /PDF

DFD Level 2 (AI Generation Process



SAMPLE CODING

App.py

```
from flask import Flask, render_template, request, send_file
import os
import uuid
import torch
from typing import Optional, List

from transformers import CLIPTokenizer, CLIPTextModel
from diffusers import StableDiffusionPipeline

from reportlab.lib.pagesizes import A4
from reportlab.pdfgen import canvas
from reportlab.lib.utils import ImageReader

app = Flask(__name__)

# ----------------------------
# Paths (local, offline)
# ----------------------------
MODEL_PATH = os.path.join("models", "sd15.safetensors")
CLIP_DIR = os.path.join("models", "clip-vit-large-patch14")

OUTPUT_DIR = os.path.join("static", "generated")
REPORT_DIR = os.path.join("static", "reports")
os.makedirs(OUTPUT_DIR, exist_ok=True)
os.makedirs(REPORT_DIR, Edisto=True)

LAST_REPORT_PATH = os.path.join(REPORT_DIR, "latest_report.pdf")

# ----------------------------
# Device (CPU laptop)
```

```
# ----------------------------
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
DTYPE = torch.float16 if DEVICE == "cuda" else torch.float32

# ----------------------------
# Load tokenizer + text encoder locally
# ----------------------------
tokenizer = CLIPTokenizer.from_pretrained(CLIP_DIR, local_files_only=True)
text_encoder = CLIPTextModel.from_pretrained(CLIP_DIR, local_files_only=True)

# ----------------------------
# Load SD pipeline from single-file checkpoint (local)
# ----------------------------
pipe = StableDiffusionPipeline.from_single_file(
    MODEL_PATH,
    torch_dtype=DTYPE,
    tokenizer=tokenizer,
    text_encoder=text_encoder,
    local_files_only=True,
    safety_checker=None,
    feature_extractor=None,
).to(DEVICE)

pipe.enable_attention_slicing()

BASE_NEGATIVE = (
    "text, watermark, logo, brand name, letters, words, caption, "
    "blurry, low resolution, low quality, jpeg artifacts, noisy, "
    "deformed, distorted, ugly, extra objects, multiple products, clutter, "
    "people, human, face, hands, fingers, cartoon, anime, messy background"
)

def build_prompt(product_name, audience, style, render_type, material, color_theme,
features):
```

```
    product_name = (product_name or "").strip()
    audience = (audience or "").strip()
    style = (style or "").strip()
    render_type = (render_type or "Product Photo").strip()
    material = (material or "").strip()
    color_theme = (color_theme or "").strip()
    features = (features or "").strip()

    if render_type == "Sketch":
        render_phrase = "industrial design sketch, clean line art, marker rendering"
    elif render_type == "3D Render":
        render_phrase = "high quality 3d render, product visualization"
    else:
        render_phrase = "studio product photo, realistic product photography"

    material_phrase = f"made of {material}" if material else ""
    color_phrase = f"color theme: {color_theme}" if color_theme else ""
    features_phrase = f"key features: {features}" if features else ""

    prompt = (
        f"A single {product_name} designed for {audience}. "
        f"{style} style. "
        f"{render_phrase}. "
        f"Center composition, plain white background, studio lighting, "
        f"clean minimal scene, sharp focus, high detail, "
        f"no extra items. "
        f"{material_phrase} {color_phrase} {features_phrase}"
    )

    return " ".join(prompt.split())

def quality_to_steps(quality):
    q = (quality or "Medium").lower()
    if q == "low":
```

```
        return 12
    if q == "high":
        return 24
    return 18

def generate_one(prompt, negative_prompt, steps, guidance, seed: Optional[int]):
    generator = None
    if seed is not None:
        generator = torch.Generator(device=DEVICE).manual_seed(seed)

    result = pipe(
        prompt=prompt,
        negative_prompt=negative_prompt,
        num_inference_steps=steps,
        guidance_scale=guidance,
        generator=generator
    )
    img = result.images[0]

    filename = f"{uuid.uuid4().hex}.png"
    save_path = os.path.join(OUTPUT_DIR, filename)
    img.save(save_path)

    # URL-friendly path (forward slashes)
    return f"generated/{filename}", save_path

def make_pdf_report(title: str, form_data: dict, prompt_used: str, image_files: List[str]):
    c = canvas.Canvas(LAST_REPORT_PATH, pagesize=A4)
    width, height = A4

    # Header
    c.setFont("Helvetica-Bold", 16)
    c.drawString(40, height - 50, "AI Product Design Generator - Design Report")
```

```
c.setFont("Helvetica", 10)
 c.drawString(40, height - 70, title)

y = height - 100
 c.setFont("Helvetica-Bold", 11)
 c.drawString(40, y, "Inputs")
 y -= 16

 c.setFont("Helvetica", 10)
 lines = [
    f"Product Name: {form_data.get('product_name','')}",
    f"Target Audience: {form_data.get('audience','')}",
    f"Style: {form_data.get('style','')}",
    f"Render Type: {form_data.get('render_type','')}",
    f"Material: {form_data.get('material','')}",
    f"Color Theme: {form_data.get('color_theme','')}",
    f"Features: {form_data.get('features','')}",
    f"Quality: {form_data.get('quality','')}",
    f"Seed: {form_data.get('seed','')}",
    f"Variations: {form_data.get('variations','1')}",
 ]
 for line in lines:
    c.drawString(40, y, line)
    y -= 14

 y -= 8
 c.setFont("Helvetica-Bold", 11)
 c.drawString(40, y, "Prompt Used")
 y -= 16

 c.setFont("Helvetica", 9)
 # Wrap prompt manually
 max_chars = 95
```

```
    prompt_lines = [prompt_used[i:i+max_chars] for i in range(0, len(prompt_used),
max_chars)]
    for pl in prompt_lines[:8]:
        c.drawString(40, y, pl)
        y -= 12

    y -= 10
    c.setFont("Helvetica-Bold", 11)
    c.drawString(40, y, "Generated Images")
    y -= 16

    # Add images (1 per page if needed)
    for idx, img_path in enumerate(image_files):
        if y < 250:
            c.showPage()
            y = height - 60
            c.setFont("Helvetica-Bold", 11)
            c.drawString(40, y, "Generated Images (continued)")
            y -= 16

        try:
            img = ImageReader(img_path)
            # Fit into box
            box_w, box_h = 500, 300
            c.drawImage(img, 40, y - box_h, width=box_w, height=box_h,
preserveAspectRatio=True, mask='auto')
            y -= (box_h + 14)
            c.setFont("Helvetica", 10)
            c.drawString(40, y, f"Variation {idx+1}")
            y -= 18
        except Exception:
            c.setFont("Helvetica", 10)
            c.drawString(40, y, f"(Could not embed image: {img_path})")
            y -= 18
```

```
 c.save()
@app.route("/", methods=["GET", "POST"])
def index():
    images = []
    prompt_used = ""
    error = None
    pdf_ready = False
   form_data = {
        "product_name": "",
        "audience": "",
        "style": "Minimalist",
        "render_type": "Product Photo",
        "material": "",
        "color_theme": "",
        "features": "",
        "quality": "Medium",
        "seed": "",
        "variations": "1"
    }
   if request.method == "POST":
        try:
            for k in form_data:
                form_data[k] = request.form.get(k, form_data[k])

            seed_val = None
            if form_data["seed"].strip():
                seed_val = int(form_data["seed"].strip())

            steps = quality_to_steps(form_data["quality"])
            guidance = 7.5

            prompt_used = build_prompt(
                form_data["product_name"],
                form_data["audience"],
```

```
            form_data["style"],
            form_data["render_type"],
            form_data["material"],
            form_data["color_theme"],
            form_data["features"],
        )
         variations = 1
        try:
          variations = int(form_data.get("variations", "1"))
        except:
            variations = 1
        variations = 3 if variations >= 3 else 1
         saved_image_paths = []
        for i in range(variations):
            # For variations: change seed automatically if seed provided
            current_seed = (seed_val + i) if seed_val is not None else None
            url_path, file_path = generate_one(
                prompt_used,
                BASE_NEGATIVE,
                steps,
                guidance,
                current_seed
            )
            images.append(url_path)
            saved_image_paths.append(file_path)

        # Build PDF report (latest_report.pdf)
        title = f"Generated on device: {DEVICE.upper()} | Steps: {steps} | Guidance:
{guidance}"
        make_pdf_report(title, form_data, prompt_used, saved_image_paths)
        pdf_ready = True

    except Exception as e:
        error = str(e)
```

```
        return render_template(
        "index.html",
        images=images,
        prompt_used=prompt_used,
        error=error,
        form_data=form_data,
        pdf_ready=pdf_ready
    )
@app.route("/download_report")
def download_report():
    if os.path.exists(LAST_REPORT_PATH):
        return send_file(LAST_REPORT_PATH, as_attachment=True)
    return "Report not found. Generate a design first.", 404
if __name__ == "__main__":
    app.run(debug=True)
```

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>AI Product Design Generator</title>
<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
<div id="loading overlay" class="overlay hidden">
<div class="overlay-card">
<div class="spinner"></div>
<h3>Generating Design…</h3>
<p>CPU laptops may take a few minutes. Please don’t refresh.</p>
</div>
</div>
<div class="container">
<header class="header">
```

```
<h1>AI Product Design Generator</h1>
<p class="sub">With Generative AI</p>
</header>
        {% if error %}
<div class="alert"><strong>Error:</strong> {{ error }}</div>
    {% endif %}
<div class="grid">
<section class="card">
<h2>Design Inputs</h2>
<form id="designForm" method="post">
<label>Product Name</label>
<input type="text" name="product_name" placeholder="e.g., Water Bottle"
                value="{{ form_data.product_name }}" required>
<label>Target Audience</label>
<input type="text" name="audience" placeholder="e.g., Kids, Students, Professionals"
                value="{{ form_data.audience }}" required>
<label>Design Style</label>
            {% set s = form_data.style %}
<select name="style">
<option value="Minimalist" {{ 'selected' if s=='Minimalist' else '' }}>Minimalist</option>
<option value="Futuristic" {{ 'selected' if s=='Futuristic' else '' }}>Futuristic</option>
<option value="Ergonomic" {{ 'selected' if s=='Ergonomic' else '' }}>Ergonomic</option>
<option value="Eco-friendly" {{ 'selected' if s=='Eco-friendly' else '' }}>Eco-
friendly</option>
<option value="Luxury" {{ 'selected' if s=='Luxury' else '' }}>Luxury</option>
<option value="Kids-friendly" {{ 'selected' if s=='Kids-friendly' else '' }}>Kids-
friendly</option>
</select>
<label>Render Type</label>
            {% set r = form_data.render_type %}
<select name="render_type">
<option value="Product Photo" {{ 'selected' if r=='Product Photo' else '' }}>Product Photo
(realistic)</option>
<option value="3D Render" {{ 'selected' if r=='3D Render' else '' }}>3D Render</option>
```

```
<option value="Sketch" {{ 'selected' if r=='Sketch' else '' }}>Industrial Sketch</option>
</select>
<div class="row">
<div>
<label>Material (optional)</label>
<input type="text" name="material" placeholder="e.g., stainless steel, BPA-free plastic"
                        value="{{ form_data.material }}">
</div>
<div>
<label>Color Theme (optional)</label>
<input type="text" name="color_theme" placeholder="e.g., matte black, pastel blue"
                        value="{{ form_data.color_theme }}">
</div>
</div>
<label>Extra Features (optional)</label>
<input type="text" name="features"
                    placeholder="e.g., leak-proof lid, carry handle, measurement marks"
                    value="{{ form_data.features }}">
<div class="row">
<div>
<label>Quality</label>
                     {% set q = form_data.quality %}
<select name="quality">
<option value="Low" {{ 'selected' if q=='Low' else '' }}>Low (faster)</option>
<option value="Medium" {{ 'selected' if q=='Medium' else '' }}>Medium</option>
<option value="High" {{ 'selected' if q=='High' else '' }}>High (slower)</option>
</select>
</div>
<div>
<label>Seed (optional)</label>
<input type="number" name="seed" placeholder="e.g., 1234"
                        value="{{ form_data.seed }}">
</div>
</div>
```

```
<label>Output Options</label>
                {% set v = form_data.variations %}
<select name="variations">
<option value="1" {{ 'selected' if v=='1' else '' }}>Generate 1 image</option>
<option value="3" {{ 'selected' if v=='3' else '' }}>Generate 3 variations</option>
</select>
<button id="submitBtn" type="submit">Generate Design</button>

</form>
</section>
<section class="card">
<h2>Output</h2>

            {% if images and images|length > 0 %}
                {% if images|length == 1 %}
<div class="output">
<img class="result-img" src="{{ url_for('static', filename=images[0]) }}" alt="Generated
Design">
<div class="actions">
<a class="btn-secondary" href="{{ url_for('static', filename=images[0]) }}"
download>Download Image</a>
                        {% if pdf_ready %}
<a class="btn-secondary" href="{{ url_for('download_report') }}">Download PDF
Report</a>
                        {% endif %}
</div>
<details class="details">
<summary>Show prompt used</summary>
<p class="prompt-box">{{ prompt_used }}</p>
</details>
</div>
                {% else %}
<div class="output">
<div class="gallery">
```

```
                {% for img in images %}
<a href="{{ url_for('static', filename=img) }}" target="_blank" title="Open">
<img class="thumb" src="{{ url_for('static', filename=img) }}" alt="Variation {{ loop.index
}}">
</a>
                {% endfor %}
</div>
<div class="actions">
                {% if pdf_ready %}
<a class="btn-secondary" href="{{ url_for('download_report') }}">Download PDF
Report</a>
                {% endif %}
</div>
<details class="details">
<summary>Show prompt used</summary>
<p class="prompt-box">{{ prompt_used }}</p>
</details>
<p class="note">Click any image to open in full size.</p>
</div>
            {% endif %}
        {% else %}
<p class="note">Your generated design will appear here after you click <b>Generate
Design</b>.</p>
        {% endif %}
```

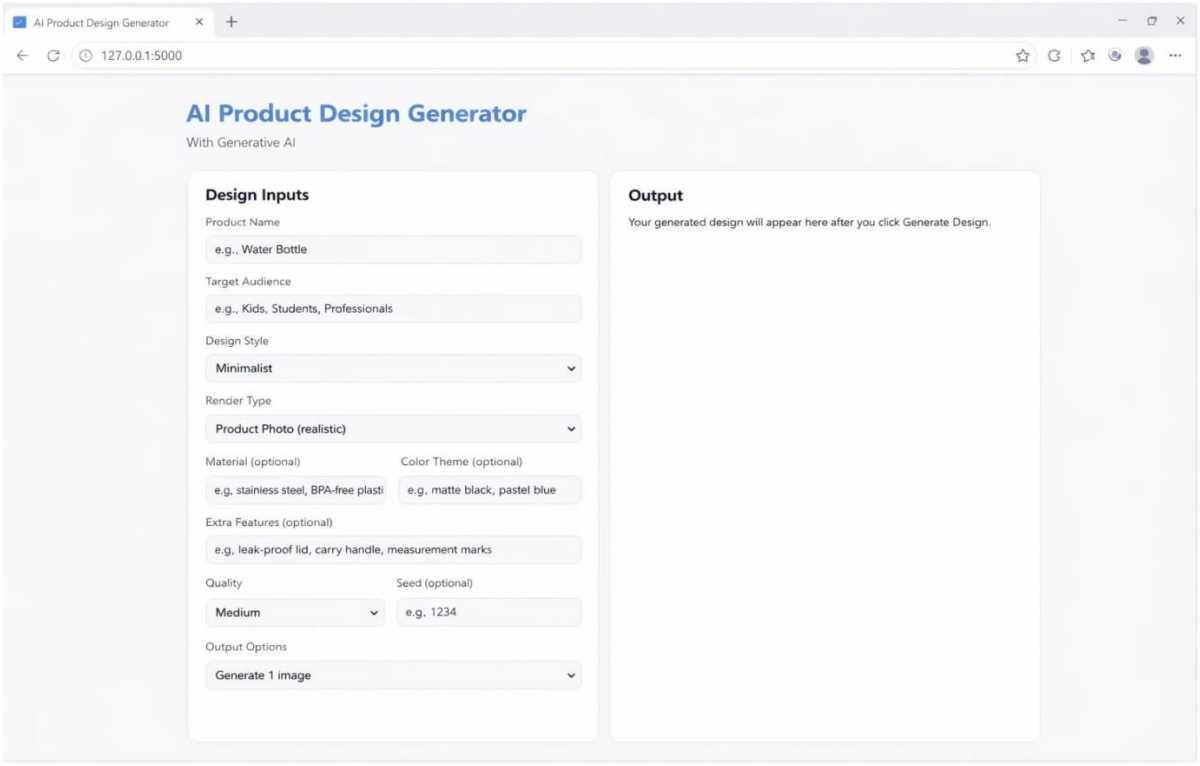
```
</section>
</div>
<footer class="footer">
<p>Copyrights@2026.</p>
</footer>
</div>
<script>



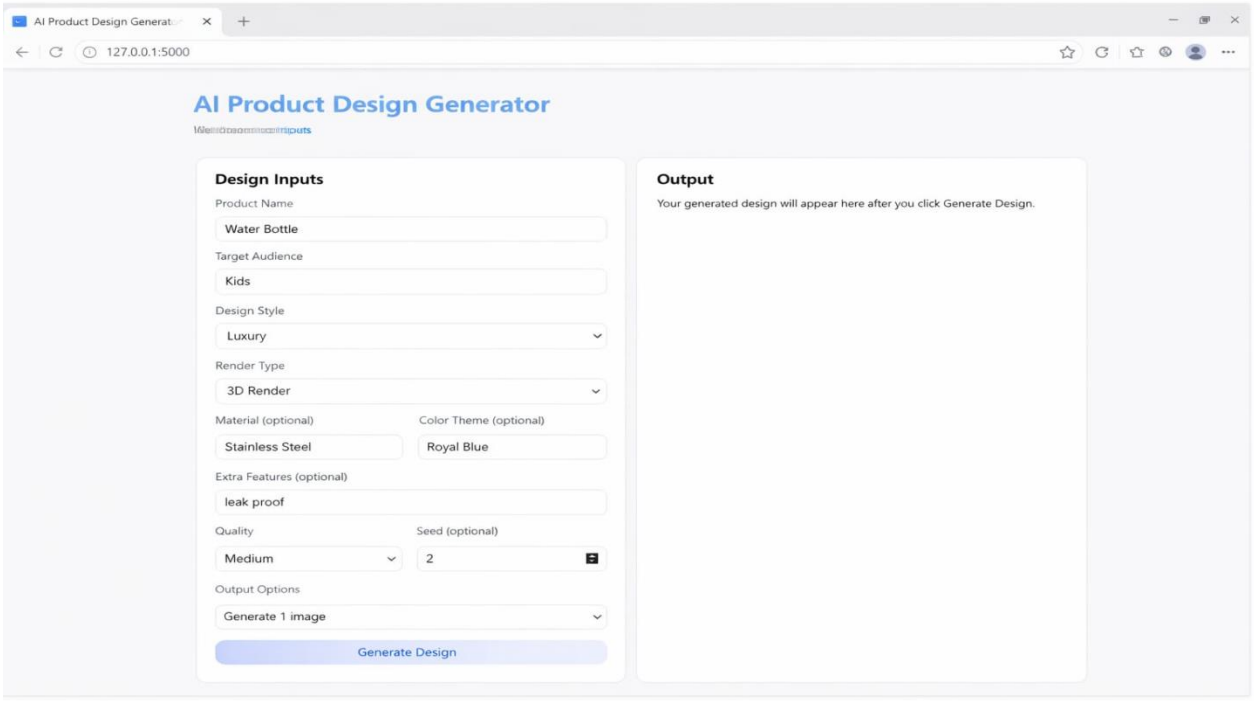
  const form = document.getElementById("designForm");
      const overlay = document.getElementById("loadingOverlay");
      const btn = document.getElementById("submitBtn");
     form.addEventListener("submit", () => {
         overlay.classList.remove("hidden");
         btn.disabled = true;
         btn.innerText = "Generating…";
      });
</script>
</body>
</html>
```

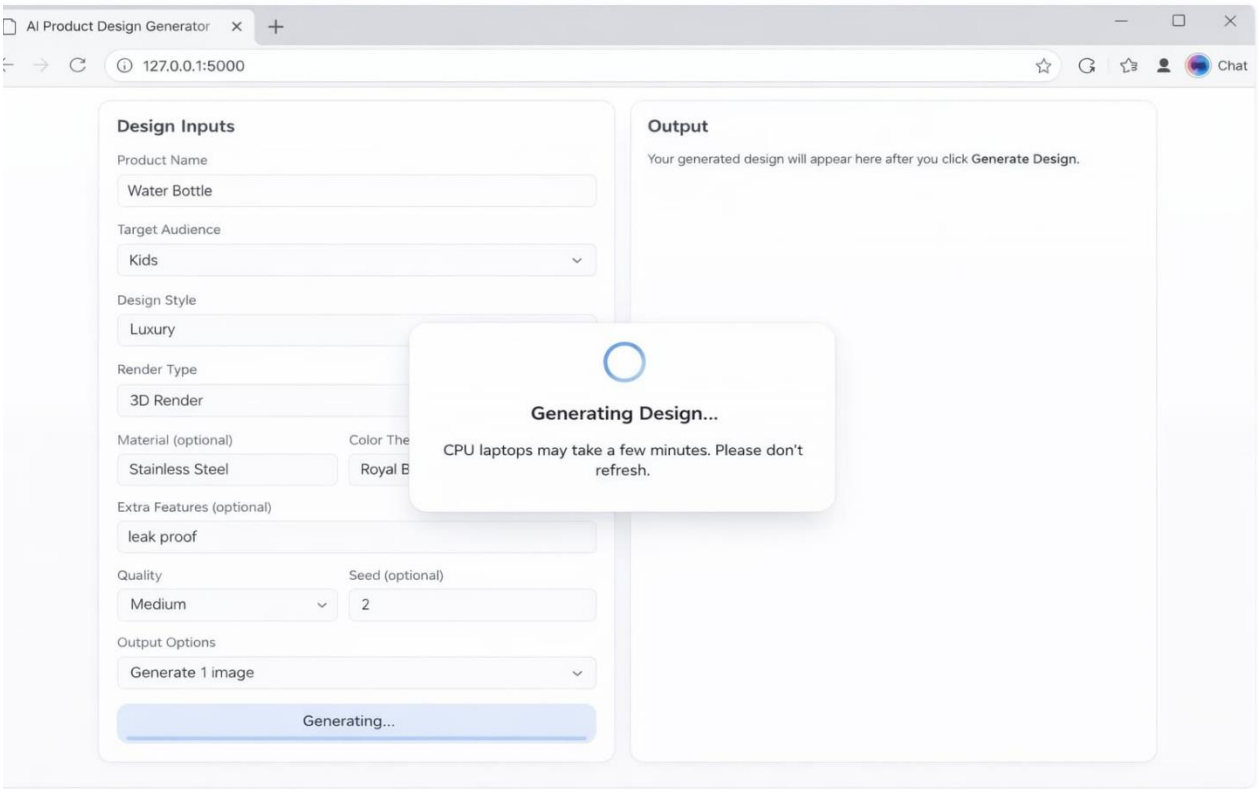
SAMPLE SCREEN

INDEX PAGE



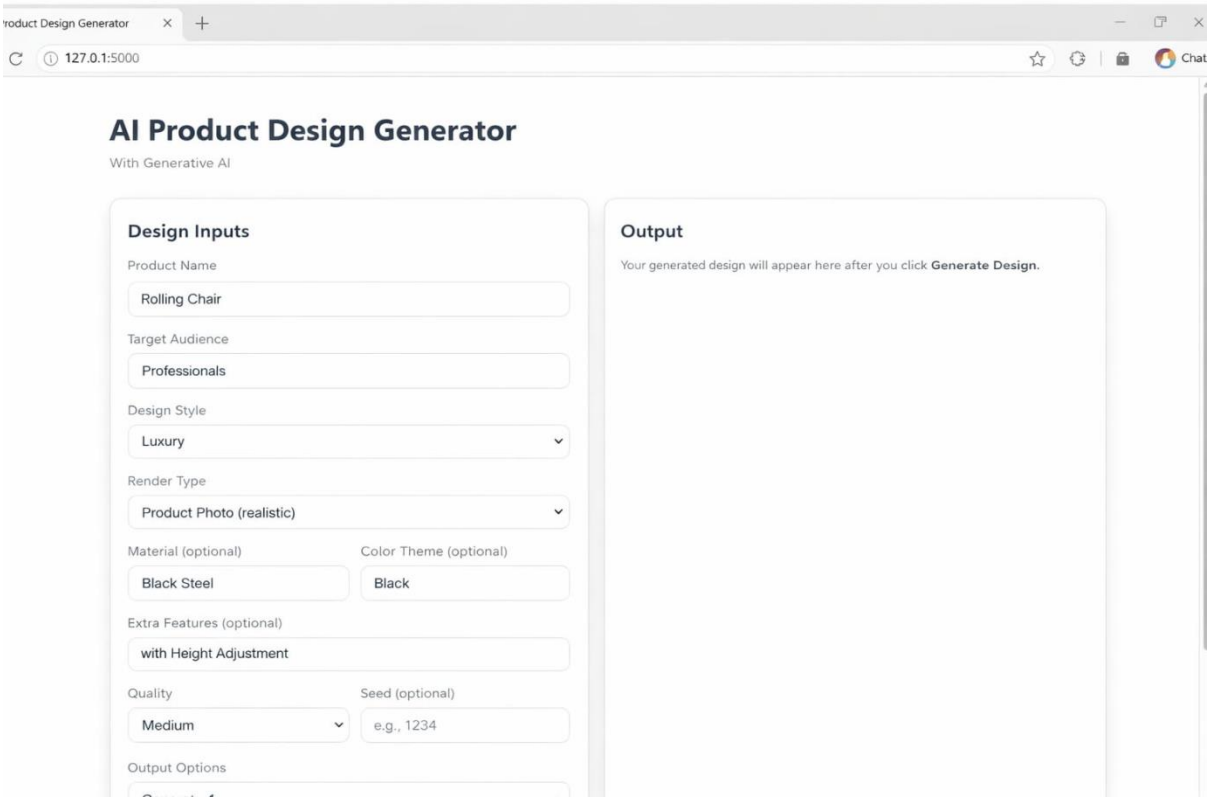
GENERATE WATER BOTTLE

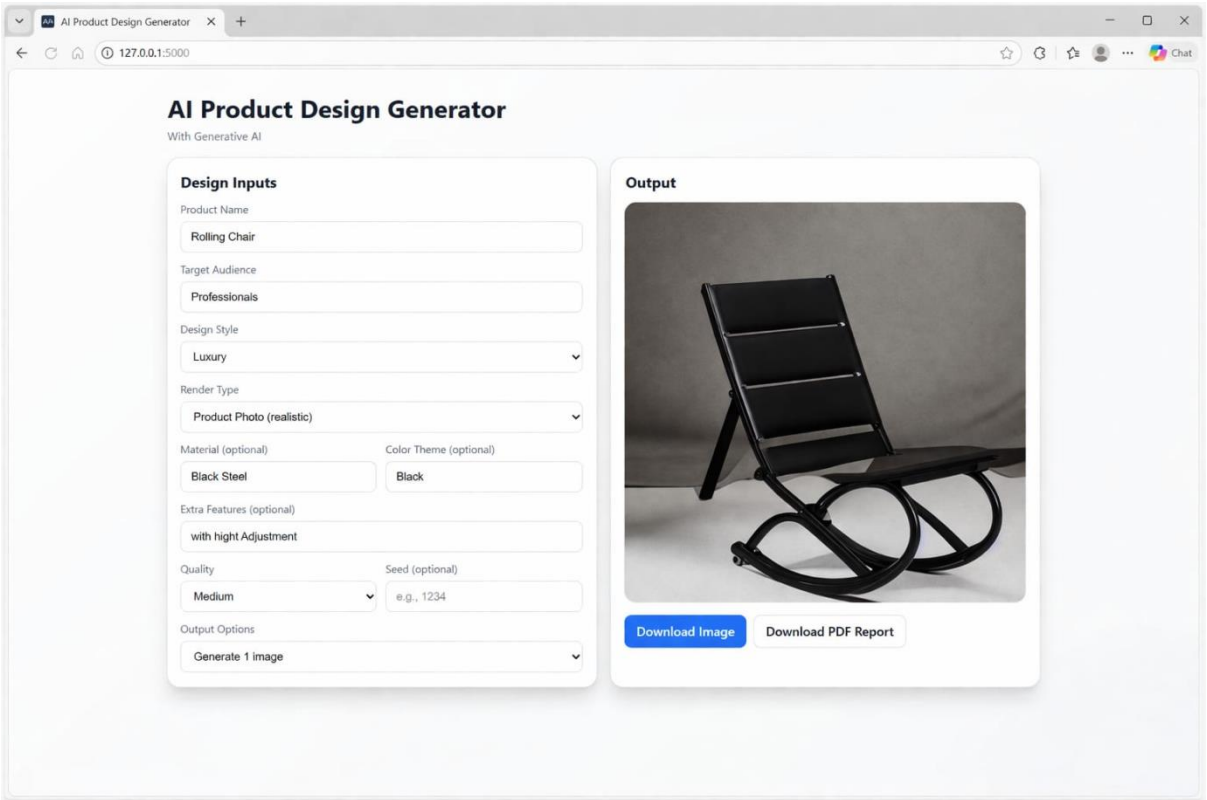


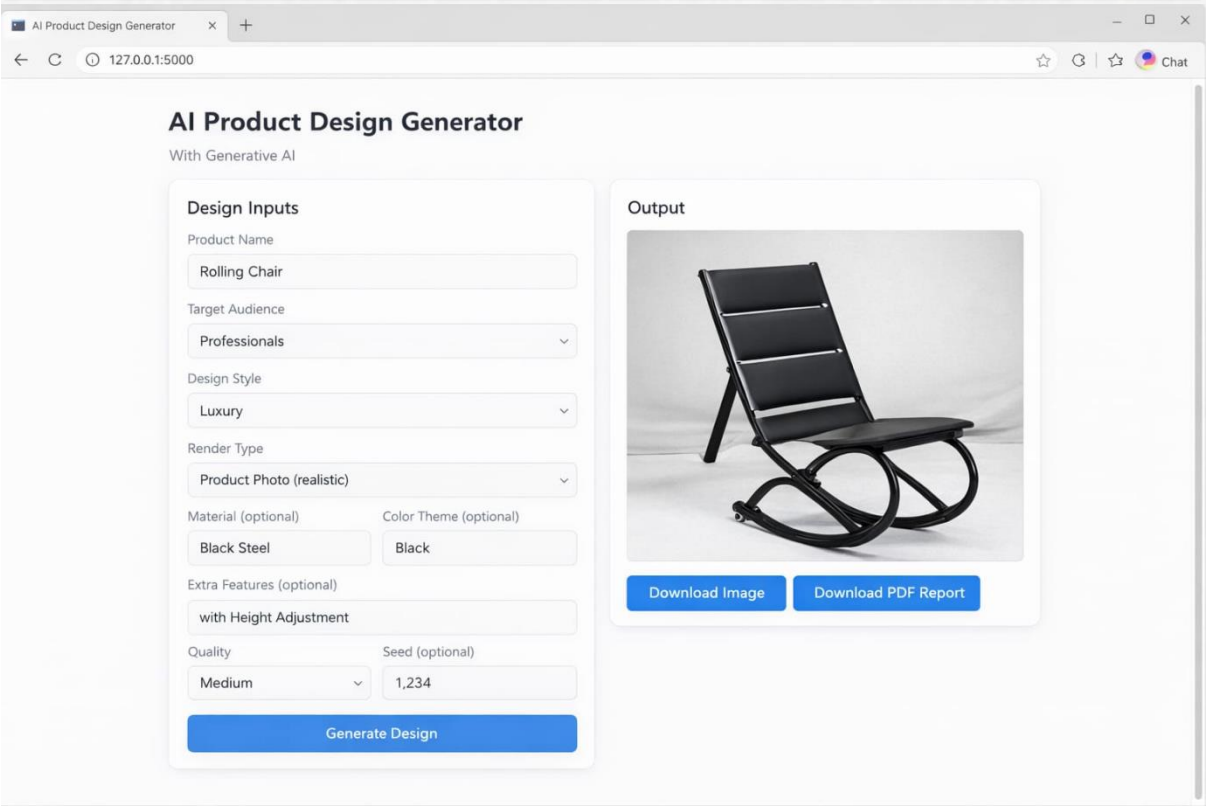




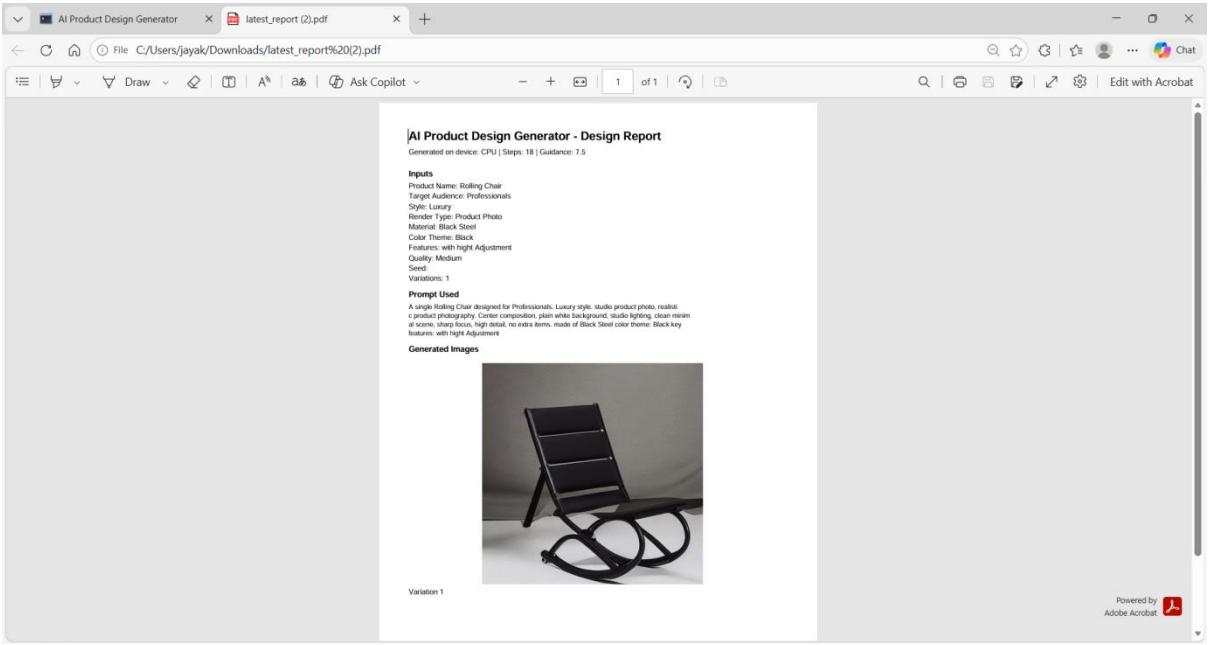
GENERATE CHAIR



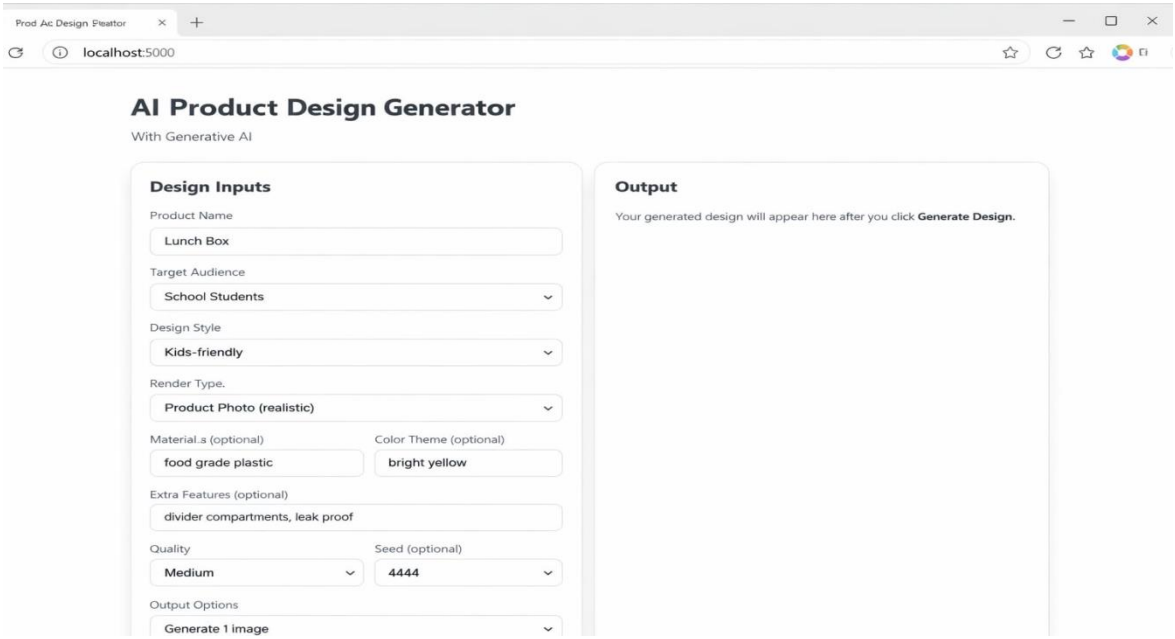

AI Product Design Generator
127.0.0.1:5000
Chat
AI Product Design Generator
With Generative AI
Design Inputs
Product Name
Rolling Chair
Target Audience
Professionals
Design Style
Luxury
Render Type
Product Photo (realistic)
Material (optional)
Black Steel
Color Theme (optional)
Black
Extra Features (optional)
with hight Adjustment
Quality
Medium
Seed (optional)
e.g., 1234
Output Options
Generate 1 image
Output
Download Image
Download PDF Report

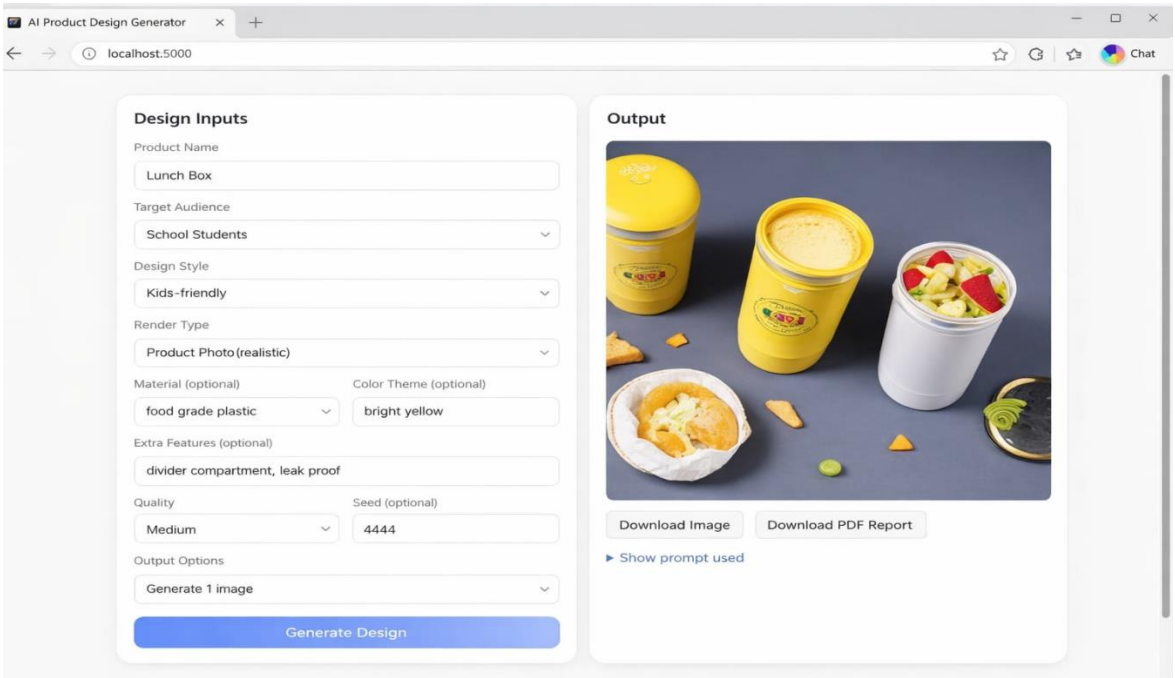

AI Product Design Generator
127.0.0.1:5000
Chat
AI Product Design Generator
With Generative AI
Design Inputs
Product Name
Rolling Chair
Target Audience
Professionals
Design Style
Luxury
Render Type
Product Photo (realistic)
Material (optional)
Black Steel
Color Theme (optional)
Black
Extra Features (optional)
with Height Adjustment
Quality
Medium
Seed (optional)
1,234
Generate Design
Output
Download Image
Download PDF Report

DOWNLOAD REPORT



GENERATE LUNCH BOX





AI Product Design Generator - Design Report

Generated on device: CPU | Steps: 18 | Guidance: 7.5

Inputs

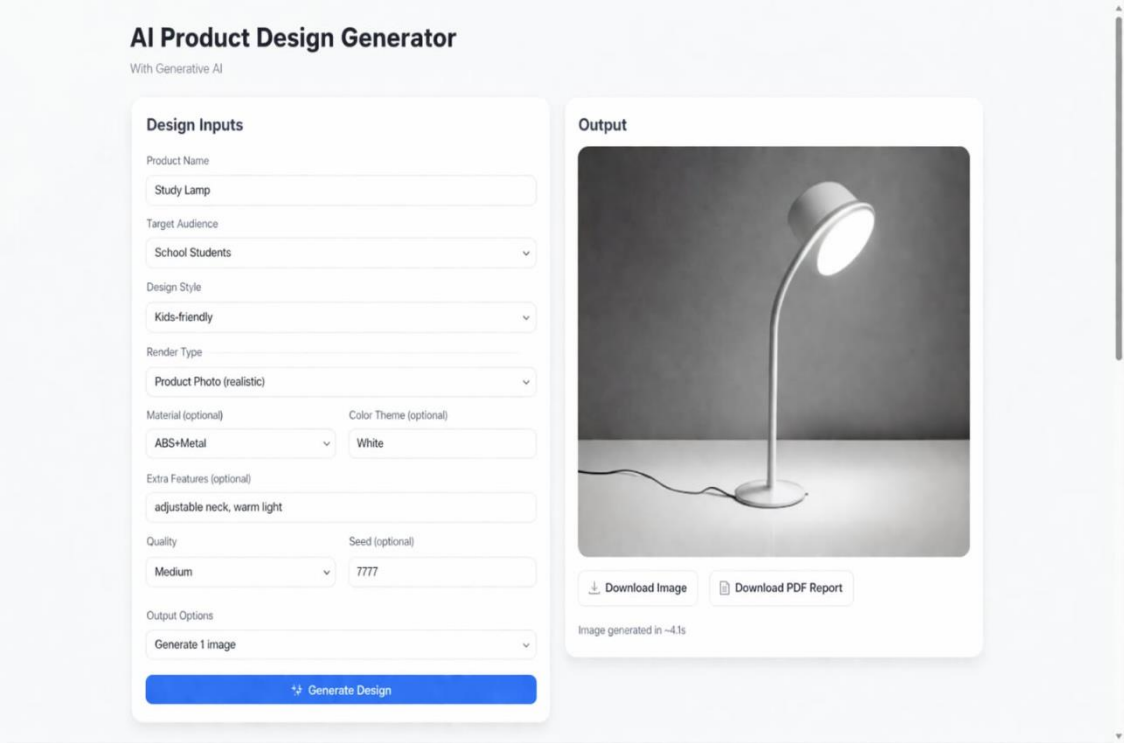
Product Name: Lunch Boix
Target Audience: School Students
Style: Kids-friendly
Render Type: Product Photo
Material: food grade plastic
Color Theme: bright yellow
Features: divider compartments, leak proof
Quality: Medium
Seed: 4444
Variations: 1

Prompt Used

A single Lunch Boix designed for School Students. Kids-friendly style. studio product photo, realistic product photography. Center composition, plain white background, studio lighting, clean minimal scene, sharp focus, high detail, no extra items. made of food grade plastic color theme: bright yellow key features: divider compartments, leak proof

Generated Images





AI Product Design Generator - Design Report

Generated on device: CPU | Steps: 18 | Guidance: 7.5

Inputs

Product Name: Study Lamp
Target Audience: School Students
Style: Kids-friendly
Render Type: Product Photo
Material: ABS+Metal
Color Theme: White
Features: adjustable neck, warm light
Quality: Medium
Seed: 7777
Variations: 1

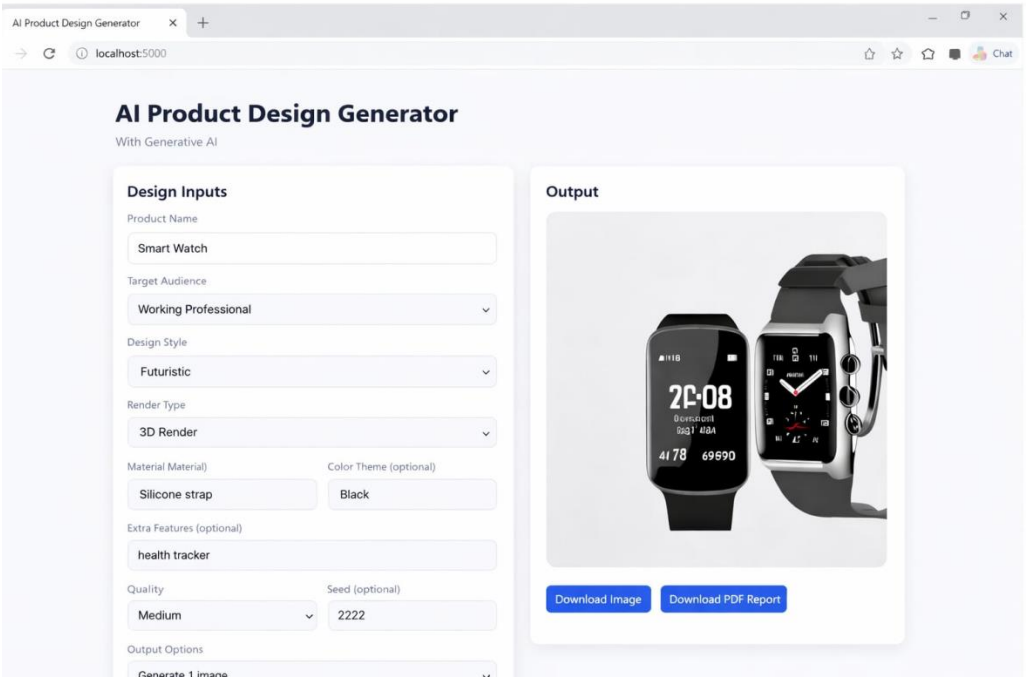
Prompt Used

A single Study Lamp designed for School Students. Kids-friendly style. studio product photo, realistic product photography. Center composition, plain white background, studio lighting, clean minimal scene, sharp focus, high detail, no extra items. made of ABS+Metal color theme: White key features: adjustable neck, warm light

Generated Images



Variation 1



AI Product Design Generator - Design Report

Generated on device: CPU | Steps: 18 | Guidance: 7.5

Inputs

Product Name: Smart Watch
Target Audience: Working Professional
Style: Futuristic
Render Type: 3D Render
Material: Silicone strap
Color Theme: Black
Features: health tracker
Quality: Medium
Seed: 2222
Variations: 1

Prompt Used

A single Smart Watch designed for Working Professional. Futuristic style. high quality 3d rende r, product visualization. Center composition, plain white background, studio lighting, clean mi nimal scene, sharp focus, high detail, no extra items. made of Silicone strap color theme: Blac k key features: health tracker

Generated Images



Variation 1